

POLITECNICO
MILANO 1863

Best Bike Paths – DD Document

DD document for the
Software Engineering 2 course assignment

Authors: **Alessandro Burato, Gabriele Crimella, Davide Faccini**

Teacher: Elisabetta Di Nitto
Academic Year: 2025-26

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms, Abbreviations . . . 1
1.4 Reference Documents . . . 2
1.5 Document Structure . . . 2

2 Architectural Design **3**
2.1 Overview . . . 3
2.2 Component View . . . 3
2.2.1 Mobile . . . 3
2.2.2 Backend . . . 3
2.3 Deployment View . . . 3
2.4 Runtime View . . . 3
2.5 Runtime View . . . 3
2.5.1 User Login . . . 3
2.5.2 User Registration . . . 5
2.5.3 User Logout . . . 6
2.5.4 Manual Trip Recording . . . 8
2.5.5 Automated Trip Recording . . . 10
2.5.6 Personal Trip Storage . . . 12
2.5.7 Path Search . . . 14
2.5.8 Browsing of Published Paths . . . 16
2.5.9 Obstacles Detection . . . 17
2.6 Component Interfaces . . . 19
2.7 Styles and Patterns . . . 19
2.8 Other Design Decisions . . . 19

3 User Interface Design **21**

4 Requirements Traceability **23**

5 Implementation, Integration and Test Plan **25**

5.1 Overview and Implementation Plan 25

5.2 Features identification 25

5.3 Integration Strategy 25

5.4 System Testing Strategy 25

6 Effort Spent **27**

7 References **29**

List of Figures **31**

List of Tables **33**

1 Introduction

1.1. Purpose

The purpose of this document is to serve as a detailed descriptor of the BBP project, to be used by actual developers and as an actual agreement between customer and contractors: it will contain detailed informations about the architecture of the system, its full functionalities, constraints and it will allow the customer to validate the integrity and completeness of said system. With this DD, the customer will also be able to validate all of the requirements, to verify if the system finally meets the expectations.

1.2. Scope

The Best Bike Paths (BBP) project, initiated by a leading bike association, aims to be an advanced support system for the cycling community designed to thrive within a transport landscape dominated by cars.

The core idea is to foster a collaborative digital environment where cyclists can actively record, consult, and publish both their own and other users' riding paths.

This platform will serve as the definitive centralized resource for finding highly recommended routes tailored precisely to the user's specific experience, preferences, and safety considerations.

Ultimately, the system aims to significantly enhance the quality of the cycling experience by providing verified, community-rated routes, thereby connecting riders and making high-quality path discovery universally accessible

1.3. Definitions, Acronyms, Abbreviations

- BBP: Best Bike Paths
- RASD: Requirements Analysis and Specification Document
- DD: Design Document

1.4. Reference Documents

- Specification document assignment
- IEEE standard documentation for DD
- Materials for the Software Engineering 2 course made available on WeBeep

1.5. Document Structure

The document is organized in seven different chapters that will cover every detail on the actual design of our project:

- **Chapter 1 - Introduction:** this chapter shows the aim of the doucment, what it is needed to know before to actually start the read of the whole text and restates the scope of the project.
- **Chapter 2 - Architectural Design:** this chapter shows all of the main components of the whole system, including architectural choices and patterns used.
- **Chapter 3 - User Interface Design:** this chapter shows and explains how the mockup pages we created integrate all of the implemented functionalities.
- **Chapter 4 - Requirements Traceability:** this chapter shows how we ensured that every requirement needed to the system to work is being satisfied with relative design choices.
- **Chapter 5 - Implementation, Integration and Test Plan:** this chapter shows how all of the components are implemented and their integration inside the system, also providing a test plan for all of their instances.
- **Chapter 6 - Effort Spent:** this chapter includes a table where the amount of hour of commitment of every participant is shown relative to its section.
- **Chapter 7 - References:** finally, the last chapter contains all of our document references used to better refine our DD.

2 | Architectural Design

2.1. Overview

2.2. Component View

2.2.1. Mobile

2.2.2. Backend

2.3. Deployment View

2.4. Runtime View

2.5. Runtime View

This section illustrates how the architectural components interact to fulfill the primary user scenarios. For each scenario, a description of the execution flow is provided, referencing the corresponding sequence of interactions.

2.5.1. User Login

This scenario describes how a registered User authenticates into the system to access private functionalities.

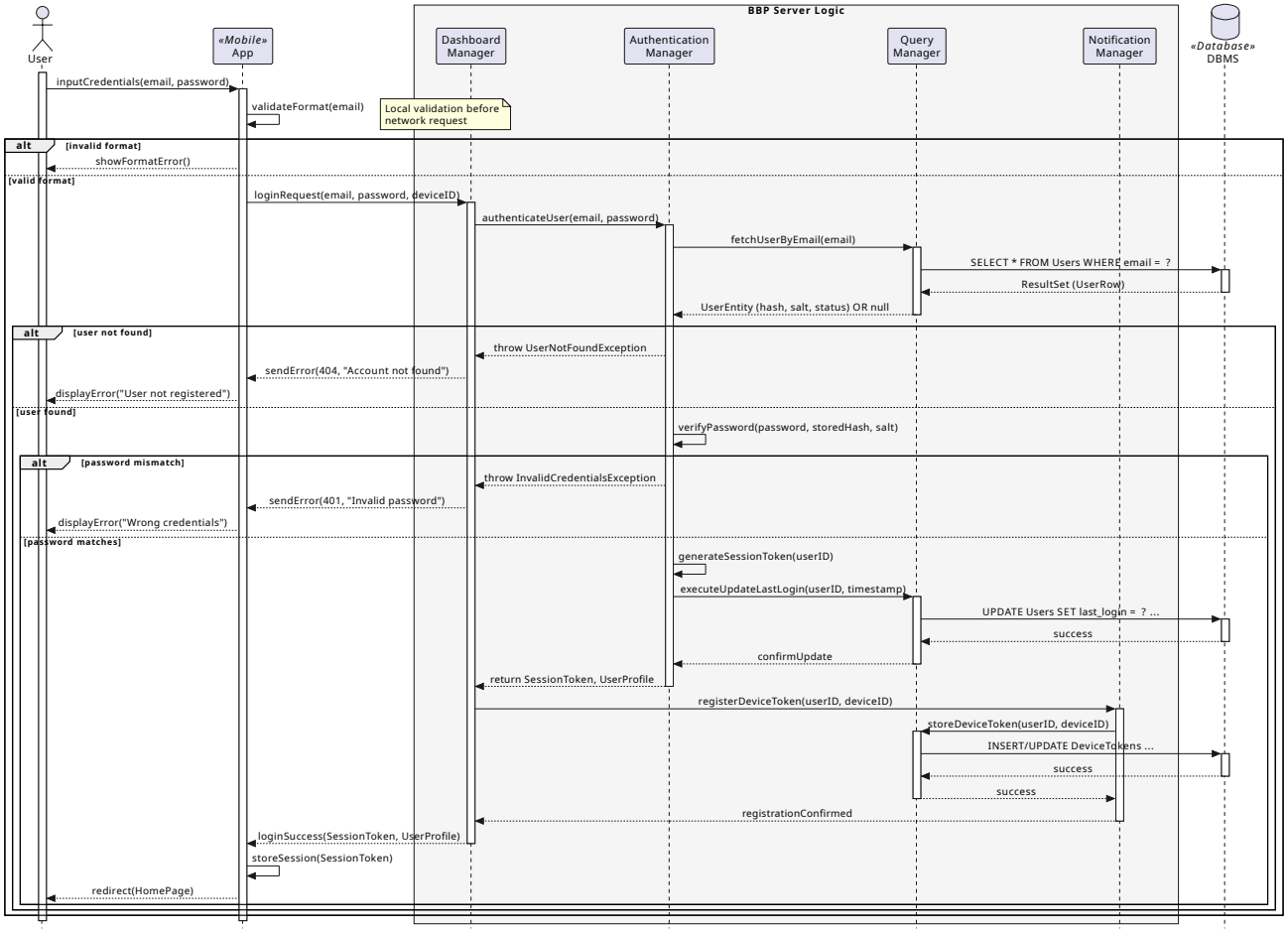


Figure 2.1: Runtime view for 'User Login'

Description of the flow: The process begins when the User inputs their credentials (email and password) into the `App`. The `App` performs a preliminary local validation (e.g., format checking) before initiating network communication. If valid, the `App` sends a request containing the credentials and the current `DeviceID` to the `Dashboard Manager`.

The `Dashboard Manager`, acting as the system entry point, delegates the identity verification to the `Authentication Manager`. To retrieve the user's stored data (such as password hash and salt), the `Authentication Manager` invokes the `Query Manager`, which executes the specific `SELECT` query against the `DBMS`.

Upon receiving the user data:

- **If the user is not found or the password hash does not match:** The `Authentication Manager` throws a specific exception. The `Dashboard Manager` catches this and returns an appropriate error message (e.g., 401 Unauthorized) to the `App`.
- **If the credentials are valid:**
  1. The `Authentication Manager` generates a new Session Token.

2. It invokes the `Query Manager` again to update the "Last Login" timestamp in the `DBMS`.
3. Control returns to the `Dashboard Manager`. Before finalizing the response, the `Dashboard Manager` coordinates with the `Notification Manager` to register the User's `DeviceID`, enabling future push notifications. The `Notification Manager` persists this association via the `Query Manager`.
4. Finally, the `Dashboard Manager` returns the Session Token and User Profile to the `App`, which stores the token and redirects the user to the homepage.

2.5.2. User Registration

This scenario details the procedure for onboarding a new User into the system. It involves a coordinated interaction between authentication mechanisms, profile management, and notification services to ensure a complete account setup.

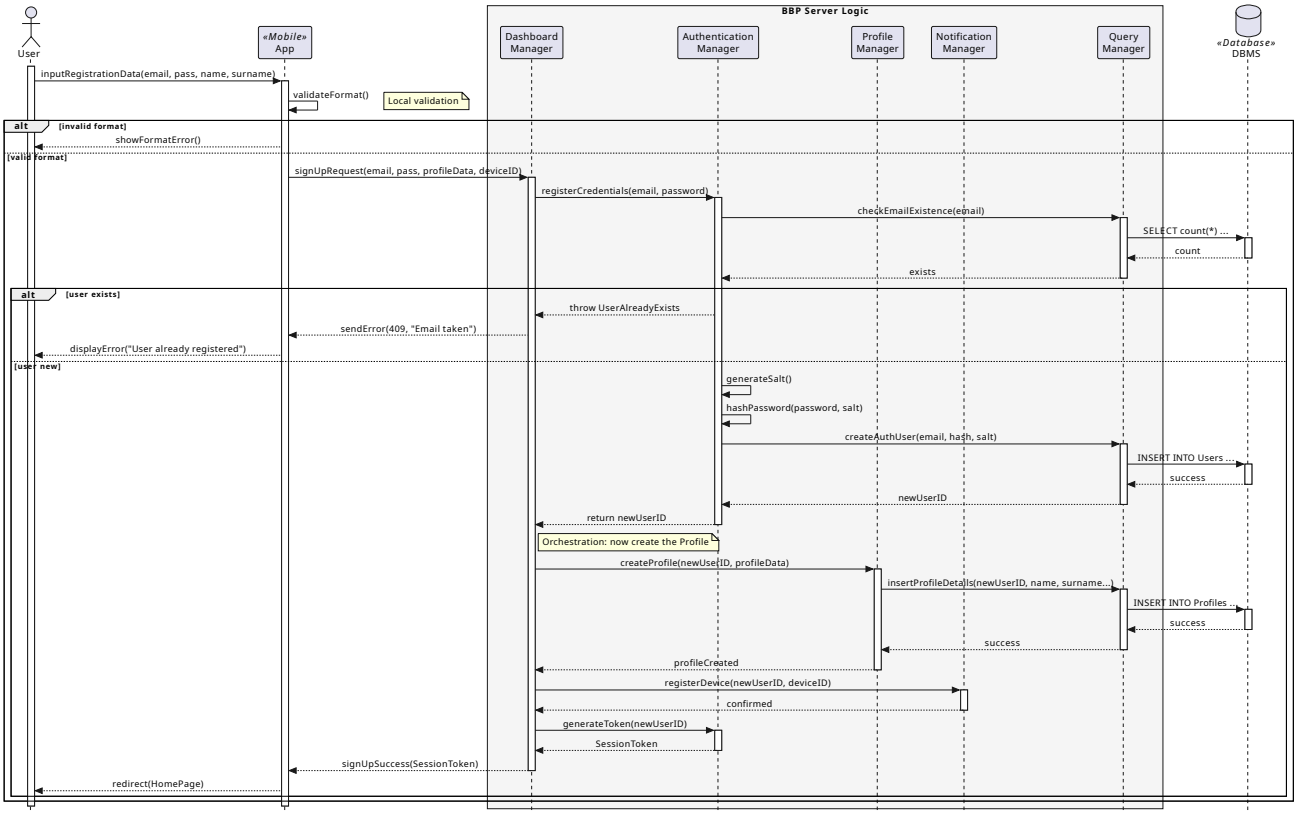


Figure 2.2: Runtime view for 'User Registration'

Description of the flow: The process begins when the User submits their registration data (email, password, and personal details such as name and surname) via the `App`. The `App` executes a local validation logic (e.g., password strength, email format) to filter out malformed requests.

If the data is valid, the App sends a composite request to the Dashboard Manager, including the current DeviceID.

The Dashboard Manager acts as the central orchestrator for this multi-step transaction, delegating specific responsibilities to the appropriate sub-components:

- **Step 1: Credential Verification and Creation.** The Dashboard Manager first invokes the Authentication Manager. This component uses the Query Manager to check if the provided email is already registered in the DBMS.
  - If the user exists, an exception is thrown, and the Dashboard Manager returns a conflict error (e.g., 409 Conflict) to the App.
  - If the user is new, the Authentication Manager hashes the password and instructs the Query Manager to persist the new security credentials. It then returns the new unique UserID to the Dashboard Manager.
- **Step 2: Profile Initialization.** Upon successful credential creation, the Dashboard Manager calls the Profile Manager. This component takes the personal information (name, surname, etc.) and the newly created UserID to initialize the user's business profile in the DBMS via the Query Manager. This ensures a strict separation between authentication data and user profile data.
- **Step 3: Device Registration and Auto-Login.** To complete the onboarding:
  1. The Dashboard Manager interacts with the Notification Manager to register the User's DeviceID, enabling immediate push notification capabilities.
  2. It requests a new Session Token from the Authentication Manager to allow for an "auto-login" experience, so the user does not need to re-enter credentials immediately after signing up.
  3. Finally, the Dashboard Manager aggregates the result and sends the Session Token back to the App, which redirects the User to the homepage.

2.5.3. User Logout

This scenario illustrates the secure termination of a User's session. It is designed as a symmetric operation to the login process, ensuring that both the security context (session validity) and the operational context (notification delivery) are properly dismantled.

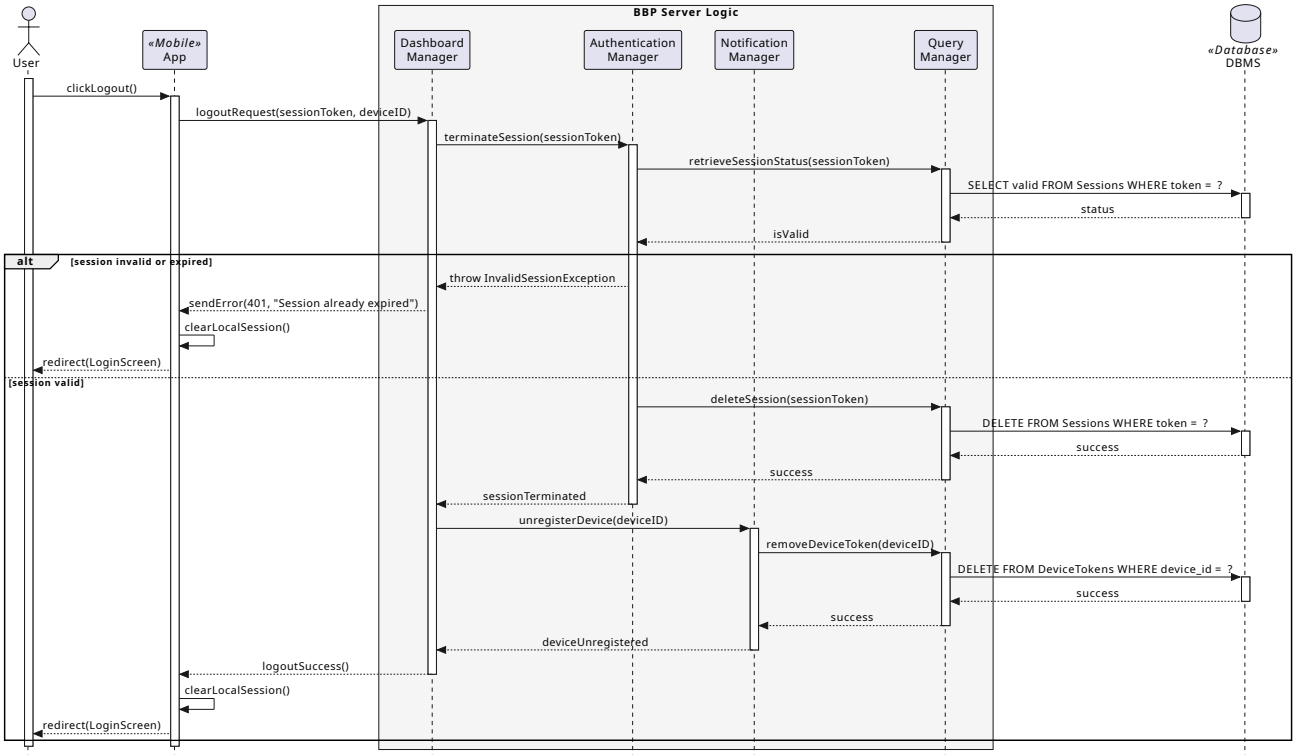


Figure 2.3: Runtime view for 'User Logout'

Description of the flow: The process is initiated when the User selects the logout option within the `App`. The `App` constructs a termination request containing the current Session Token and the Device ID, transmitting it to the `Dashboard Manager`.

The `Dashboard Manager` orchestrates the teardown of the user's active context through two main phases:

- **Phase 1: Session Invalidation.** The `Dashboard Manager` first instructs the `Authentication Manager` to terminate the session.
  - The `Authentication Manager` queries the `DBMS` via the `Query Manager` to verify the token's validity.
  - If valid, the token is physically removed or marked as expired in the database. This step is critical to prevent session replay attacks. If the session is already invalid (e.g., expired), the system handles the exception gracefully, ensuring the client is still forced to a logged-out state.
- **Phase 2: Notification Deregistration.** To ensure no further personal alerts are sent to the device, the `Dashboard Manager` coordinates with the `Notification Manager`.
  - The `Notification Manager` invokes the `Query Manager` to remove the association between the user and the specific `DeviceID` in the `DBMS`. This mirrors the registration

step performed during login/signup.

Upon the successful completion of server-side cleanup, the `Dashboard Manager` sends a confirmation to the `App`. The `App` then clears all local storage (cached tokens, user data) and redirects the User to the login screen.

2.5.4. Manual Trip Recording

This scenario describes the process by which a Registered User manually reconstructs a trip by inputting a sequence of nodes and metadata.

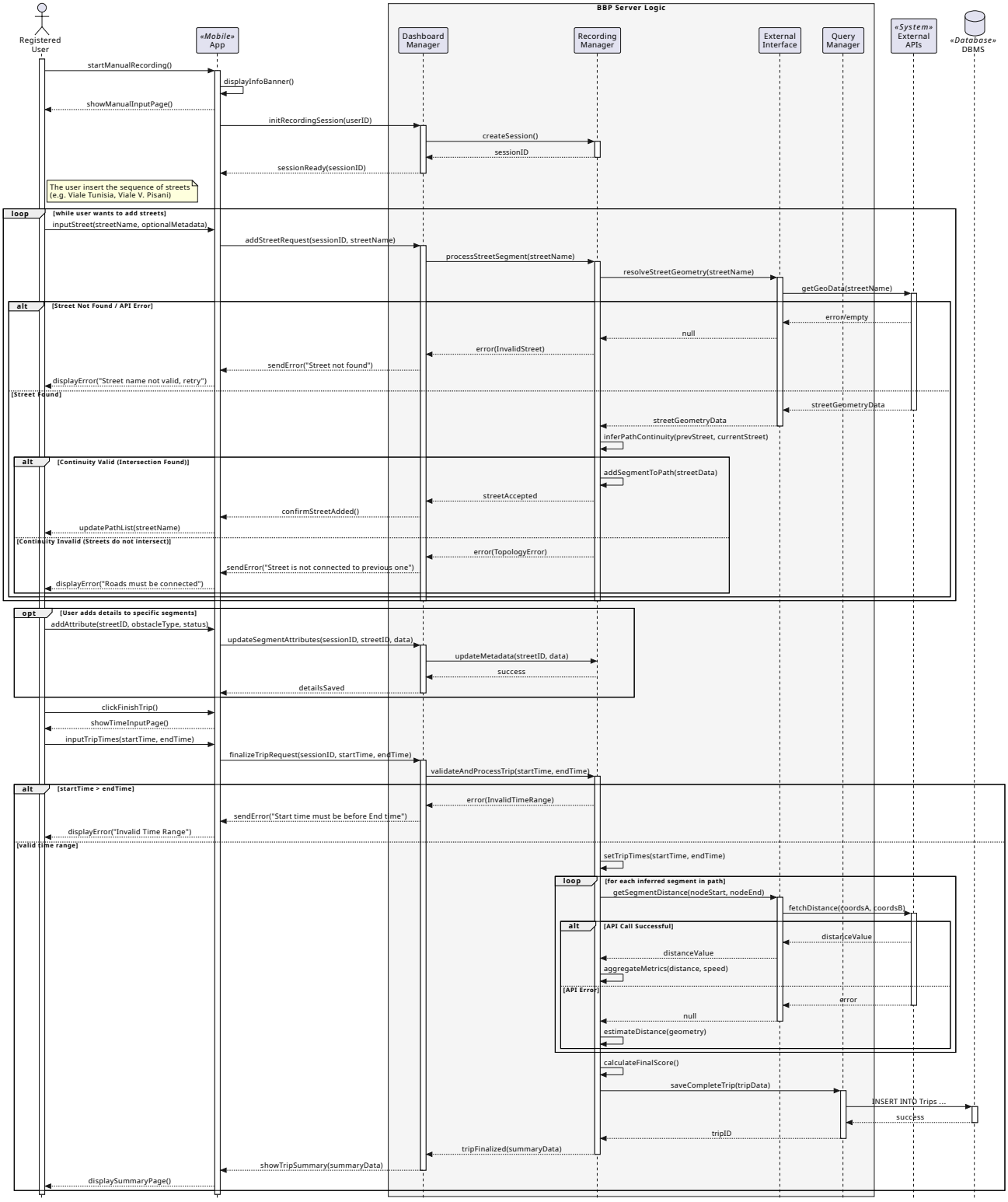


Figure 2.4: Runtime view for 'Manual Trip Recording'

Description of the flow: The interaction begins when the User selects the "Manual Mode" on the `App`. After acknowledging the introductory information, the process unfolds in three distinct phases managed by the `Recording Manager` via the `Dashboard Manager`:

- **Phase 1: Path Reconstruction by Streets.** The User inputs the sequence of **street names** traversed during the trip.
  - For each inserted street, the `Recording Manager` interacts with the `External Interface` to validate the street's existence and retrieve its geometry.
  - Crucially, the system performs a **topology check** (inference) to ensure the new street physically intersects or connects with the previous one.
  - Upon successful validation of continuity, the street is added to the path list. The User can optionally associate metadata (e.g., obstacles or road status) to specific street segments. If a street is not found or not connected, the system prompts the User for correction.
- **Phase 2: Temporal Definition.** Once the sequence of streets is defined, the User concludes the session and manually inputs the **start and end times** of the trip.
  - The `Recording Manager` validates that the start time strictly precedes the end time. Invalid ranges trigger an error response, prompting the User to re-enter the temporal data.
- **Phase 3: Data Enrichment and Persistence.** Before saving, the system automatically enhances the reconstructed path:
  1. The `Recording Manager` iterates through the inferred segments (derived from street intersections) and invokes the `External Interface` to request accurate distance metrics from the `External APIs`.
  2. If the external service fails (e.g., timeout), the system attempts to estimate the distance based on the retrieved geometry to ensure process resilience.
  3. Once all metrics are aggregated (calculating total distance and average speed based on the time range), the `Recording Manager` uses the `Query Manager` to persist the complete Trip object into the `DBMS`.
  4. Finally, a summary of the recorded activity is returned to the `App` for display.

2.5.5. Automated Trip Recording

This scenario details the "hands-free" recording mode, where the system leverages device sensors to detect activity and automates data collection. It features a complex flow involving local hardware interaction and concurrent server-side processes to enrich trip data in real-time.

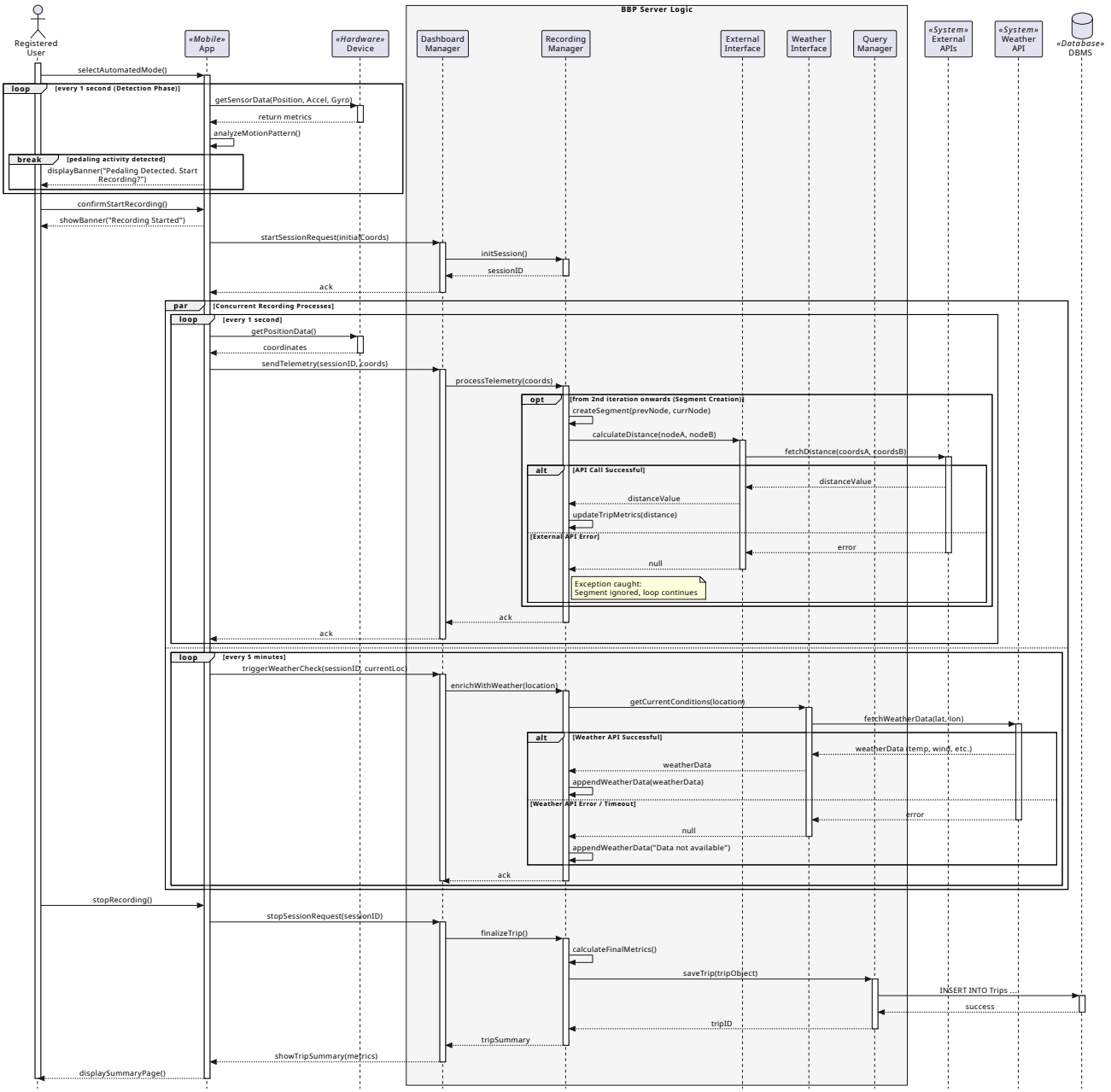


Figure 2.5: Runtime view for 'Automated Trip Recording'

Description of the flow: The workflow is initiated when the User activates "Automated Mode" on the `App`. The process is structured into three logical phases, distinguishing between local detection and server-side processing:

- **Phase 1: Motion Detection and Initialization.**
  - The `App` enters a monitoring loop, periodically querying the local `Device` for sensor metrics (accelerometer, gyroscope, and position).
  - Upon detecting a pattern consistent with cycling, the `App` prompts the User for confirmation.

  - Once confirmed, the `App` sends an initialization request to the `Dashboard Manager`, which instructs the `Recording Manager` to create a new active session.

- **Phase 2: Concurrent Recording Loops.** During the active trip, the system orchestrates two parallel execution threads to optimize data accuracy and API usage:

  1. **High-Frequency Telemetry Loop (every 1s):** The `App` streams GPS coordinates to the `Recording Manager`. For every new segment detected, the system invokes the `External Interface` to fetch precise distance metrics from `External APIs`. Exception handling is in place to skip segments if the external service is unreachable, ensuring continuous recording.

  2. **Low-Frequency Environmental Loop (every 5 mins):** To enrich the trip data without exhausting API quotas, the `Recording Manager` periodically triggers a weather check. It uses the `Weather Interface` to retrieve conditions from the `Weather API`. If the API fails, the system gracefully defaults to a "Data not available" state rather than interrupting the flow.

- **Phase 3: Termination and Persistence.**

  - When the User manually stops the recording, the `App` signals the termination to the `Dashboard Manager`.

  - The `Recording Manager` finalizes the calculations (aggregating distances and speeds) and invokes the `Query Manager` to persist the complete Trip object into the `DBMS`.

  - Finally, a summary of the performance metrics is returned to the User.

2.5.6. Personal Trip Storage

This scenario illustrates the management of the User's personal history of trips. It covers the retrieval of stored data and, crucially, the mechanism for toggling the visibility of a trip between "Private" (visible only to the owner) and "Public" (searchable by the community).

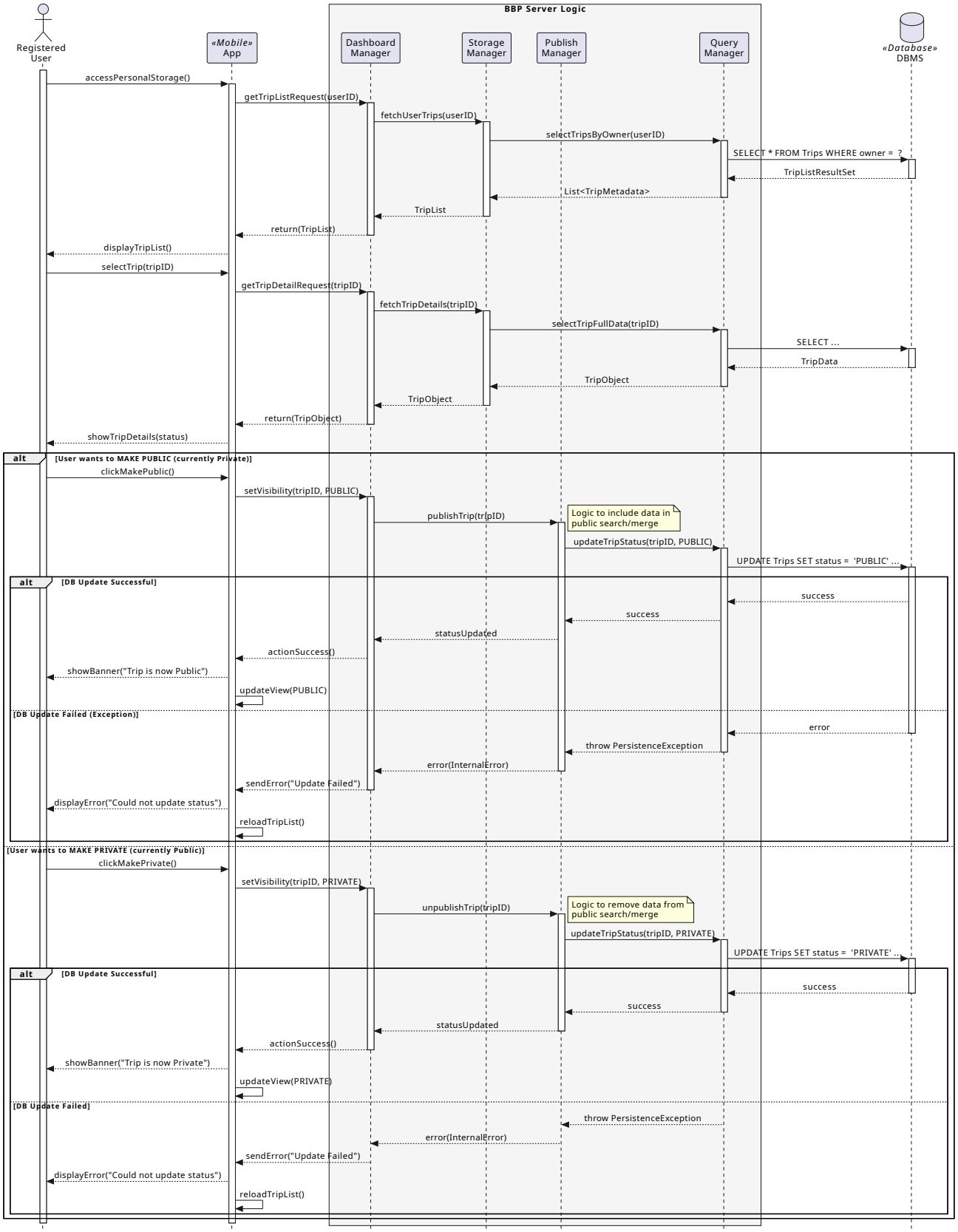


Figure 2.6: Runtime view for 'Personal Trip Storage'

Description of the flow: The interaction is structured into two main logical phases: data

retrieval and status modification.

- **Phase 1: Retrieval of Trip Data.** When the User accesses the storage section, the `Dashboard Manager` delegates the request to the `Storage Manager`. This component interacts with the `Query Manager` to fetch the metadata list of trips owned by the specific User from the `DBMS`. A similar flow applies when the User requests the full details of a specific trip.
- **Phase 2: Visibility Management (Publication/Unpublication).** The core complexity lies in changing a trip's status. When the User toggles the visibility (e.g., "Make Public" or "Make Private"):
  1. The request is routed to the `Publish Manager` (distinct from the Storage Manager). This component validates the request and invokes the `Query Manager` to execute an atomic `UPDATE` on the trip's status field in the `DBMS`.
  2. **Success Path:** If the database confirms the update, the system acknowledges the change. For a "Make Public" action, this logically implies the trip becomes available for the system's "Data Merge" algorithms. The `App` updates the view immediately.
  3. **Failure Path:** As shown in the diagram's alternative flows, if a persistence error occurs (e.g., database lock or connection failure), the system prioritizes consistency. The `Publish Manager` propagates the exception, the `Dashboard Manager` returns an error, and the `App` forces a reload of the list. This ensures the User never sees a "false positive" state where the UI shows "Public" while the DB remains "Private".

2.5.7. Path Search

This scenario illustrates the workflow for retrieving bike paths based on geographical constraints. It emphasizes the validation logic performed by the backend to ensure data consistency before executing complex search queries.

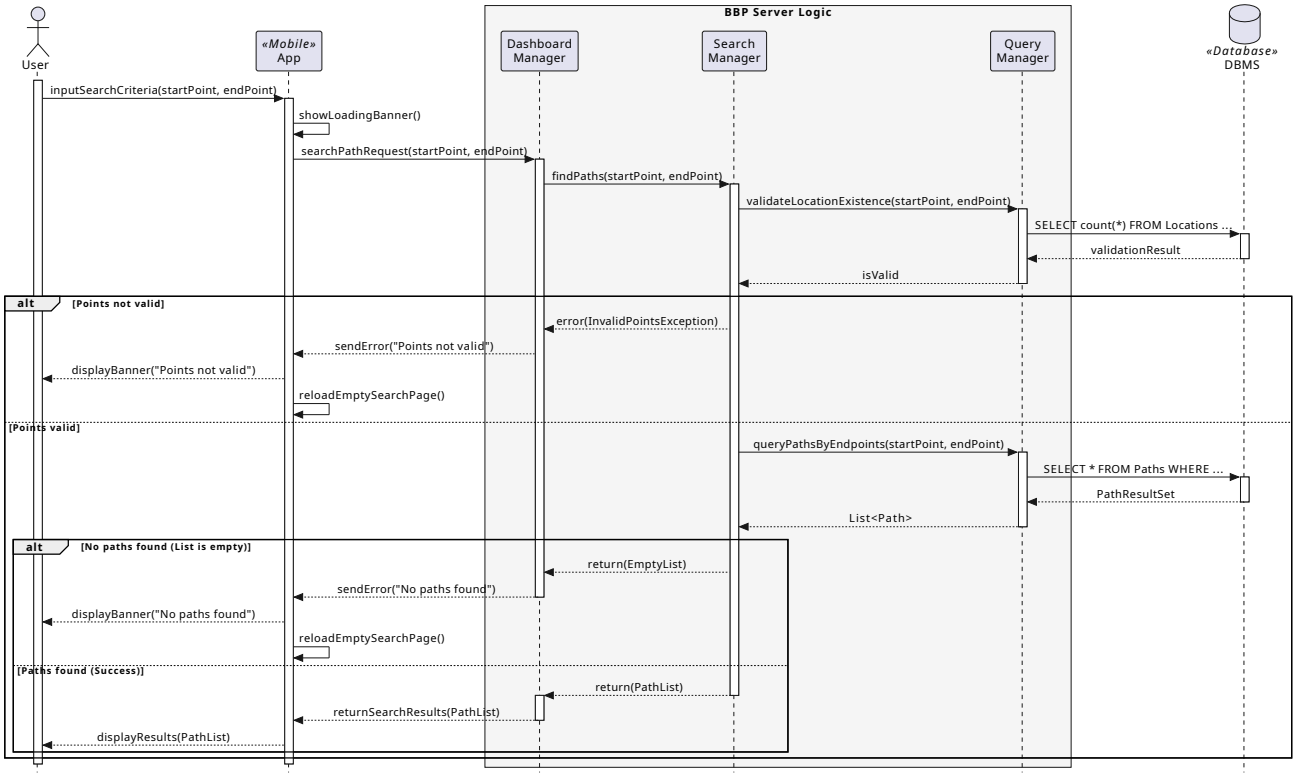


Figure 2.7: Runtime view for 'Path Search'

Description of the flow: The interaction begins when the User inputs the desired start and end points via the `App` interface. The `App` displays a loading state and forwards the search criteria to the `Dashboard Manager`.

The `Dashboard Manager` routes the request to the `Search Manager`, which orchestrates the retrieval process in two distinct logical steps to optimize resource usage:

- **Step 1: Input Validation.** Before searching for paths, the `Search Manager` invokes the `Query Manager` to verify that both the start and end points correspond to valid locations existing in the `DBMS`.
  - If one or both points are invalid, the `Search Manager` raises an exception. The `App` receives an error message (e.g., "Points not valid"), displays a specific banner to the User, and reloads the search page to allow for correction.
- **Step 2: Path Retrieval.** If the points are valid, the `Search Manager` instructs the `Query Manager` to execute the specific query to find all paths connecting the two locations.
  - **No Paths Found:** If the query returns an empty result set, the system notifies the `App`. The User is informed via a "No paths found" banner, and the interface is reset for a new search.
  - **Search Successful:** If matching paths are found, the `Query Manager` returns the

list of path objects. The `Search Manager` forwards this data through the `Dashboard Manager` to the `App`, which renders the results list for the User to browse.

2.5.8. Browsing of Published Paths

This scenario depicts the workflow allowing Users to explore the catalog of publicly available bike paths within a specific geographical area. It highlights the filtering logic that ensures only content explicitly marked as "Public" is accessible to the general user base.

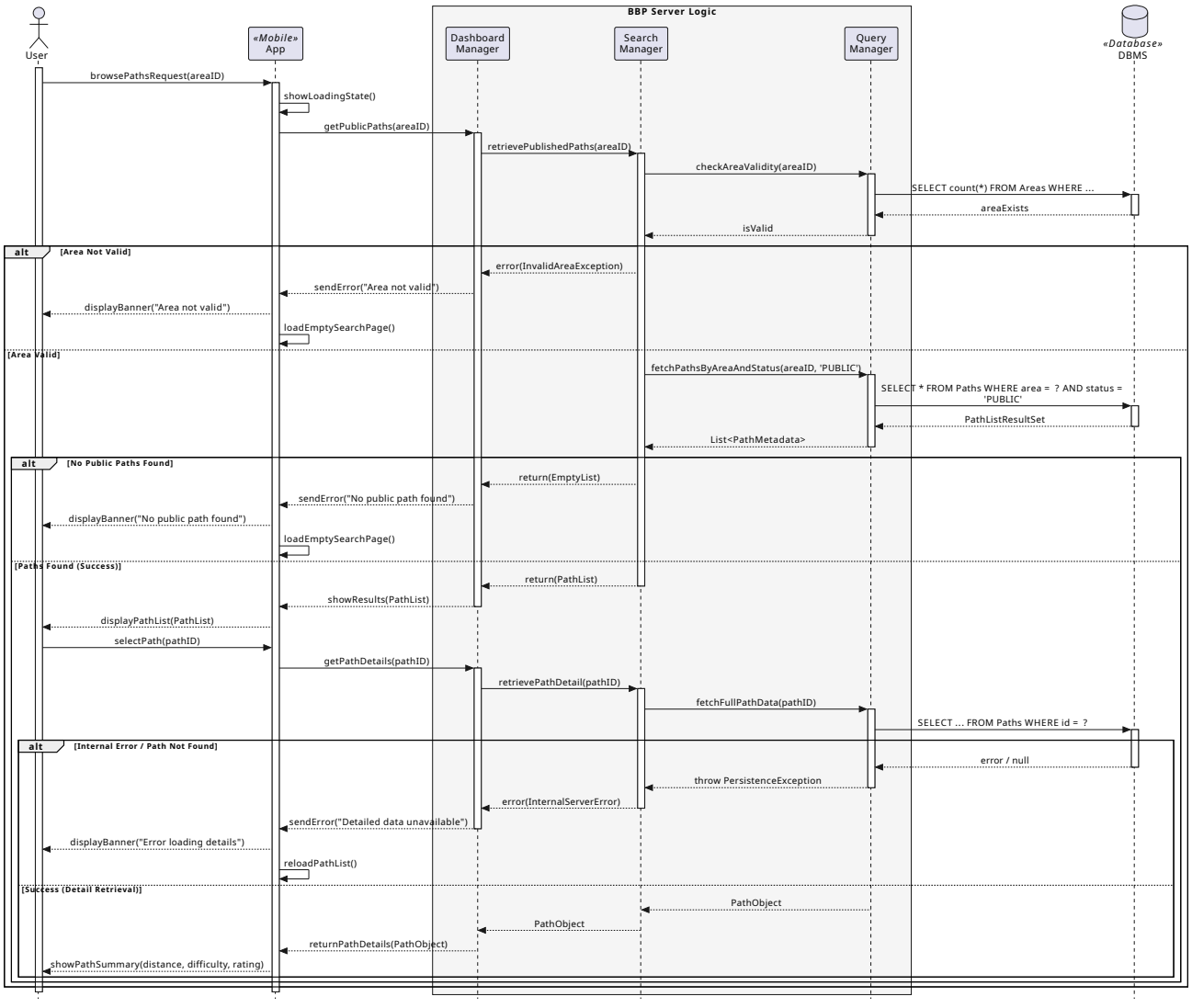


Figure 2.8: Runtime view for 'Browsing of Published Paths'

Description of the flow: The interaction begins when the User selects a geographical area of interest via the `App`. The interface displays a loading state while the request is forwarded to the `Dashboard Manager`, which delegates the operation to the `Search Manager`.

The retrieval process is orchestrated in three sequential phases to ensure data consistency and

validity:

- **Phase 1: Area Validation.** Before attempting any data retrieval, the `Search Manager` invokes the `Query Manager` to verify the existence of the requested area in the `DBMS`.
  - If the area ID is malformed or non-existent, the process terminates immediately, and the `App` prompts the User to select a valid location.
- **Phase 2: Public Paths Retrieval.** Upon successful validation, the `Search Manager` requests the list of paths associated with that area.
  - Crucially, the query executed by the `Query Manager` includes a strict filter for `status = 'PUBLIC'`. Private trips stored by other users are strictly excluded at the database level.
  - If the query returns results, the list is displayed to the User. If the area contains no public paths, a specific notification is returned.
- **Phase 3: Detail View.** The workflow concludes with the User selecting a specific path from the list.
  1. The `App` requests the full details (including metrics like difficulty and rating) for the selected ID.
  2. The `Search Manager` retrieves the complete object via the `Query Manager`.
  3. Should an internal consistency error occur (e.g., the path was deleted moments before selection), the system handles the exception by notifying the User and refreshing the list, rather than crashing the application.

2.5.9. Obstacles Detection

This scenario details the advanced telemetry analysis performed during an automated recording session. It illustrates how the system leverages hardware sensor data (accelerometer and gyroscope) to identify potential road hazards and includes a human-in-the-loop verification step to ensure data accuracy.

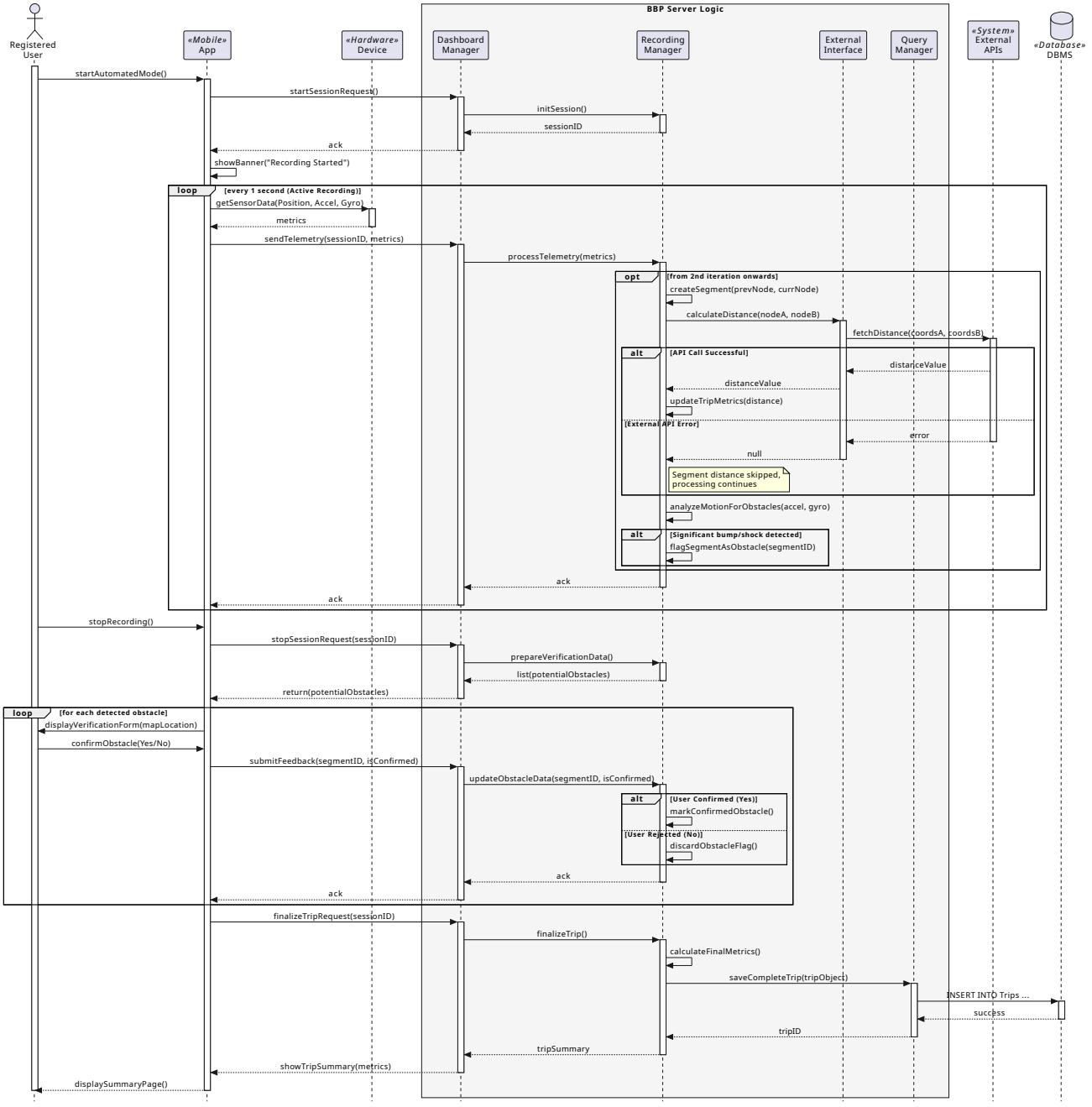


Figure 2.9: Runtime view for 'Obstacles Detection'

Description of the flow: The workflow is an extension of the Automated Recording mode. Once the recording session is initialized via the `Dashboard Manager`, the process executes in two distinct temporal stages:

- **Stage 1: Real-Time Detection and Enrichment.** During the active trip, the `App` streams sensor metrics to the `Recording Manager` at one-second intervals.

  1. **Distance Calculation:** The system invokes the `External Interface` to fetch segment lengths from `External APIs`. As depicted in the diagram, this interaction is

designed to be resilient: if the external service returns an error, the specific segment is skipped, allowing the recording loop to continue uninterrupted.

2. **Anomaly Recognition:** Simultaneously, the `Recording Manager` analyzes the accelerometer and gyroscope data. If a motion pattern consistent with a road anomaly (e.g., a pothole or bump) is detected, the current segment is internally flagged as a "Potential Obstacle".

- **Stage 2: Post-Trip User Verification.** When the User stops the recording, the system does not immediately save the trip but initiates a verification sequence.
  1. The `Recording Manager` returns the list of flagged segments to the `App`.
  2. The `App` presents these locations to the User, asking for confirmation. The User iterates through the list, confirming ("Yes") or rejecting ("No") the presence of an obstacle.
  3. The `Dashboard Manager` collects this feedback and updates the segment data accordingly, filtering out false positives.
- **Stage 3: Persistence.** Once the verification loop is complete, the `Recording Manager` calculates the final trip metrics and invokes the `Query Manager` to persist the validated Trip object (including confirmed obstacles) into the `DBMS`.

2.6. Component Interfaces

2.7. Styles and Patterns

2.8. Other Design Decisions

3 | User Interface Design

4 | Requirements Traceability

5 | Implementation, Integration and Test Plan

5.1. Overview and Implementation Plan

5.2. Features identification

5.3. Integration Strategy

5.4. System Testing Strategy

6 | Effort Spent

7 | References

List of Figures

2.1 Runtime view for 'User Login' 4
2.2 Runtime view for 'User Registration' 5
2.3 Runtime view for 'User Logout' 7
2.4 Runtime view for 'Manual Trip Recording' 9
2.5 Runtime view for 'Automated Trip Recording' 11
2.6 Runtime view for 'Personal Trip Storage' 13
2.7 Runtime view for 'Path Search' 15
2.8 Runtime view for 'Browsing of Published Paths' 16
2.9 Runtime view for 'Obstacles Detection' 18

List of Tables